

Activity VIII: Secure Software/ Simple Web Server

Instructor : Kerk Piromsopa, Ph.D

Some problems and the examples are taken/adopt from Google Web Security

Programming Problem Set . Some parts have been modified to serve the purpose.

Overview

We will use JAVA in this example. Technically, you may do it in your favorite programming languages. We will learn the importance of Secure by Design by designing a simple web server.

The provided SimpleWebServer is a single thread/session web server. While serving a purpose, it is far from being secure. A connection to the server will block others from accessing and may result in **denial of service**. In addition, there is no access control at all.

This makes it vulnerable to **elevation of privileges, information disclosure and tampering**. There is also no log for non-repudiation.

You are now in charge of securing it. Please apply the knowledge of secure by design to secure this SimpleWebServer. Keep in mind that this is an educational assignment. Don't try to use it in any real production. It takes more effort than you can imagine to secure a software.

Exercises

Q1.

Q1.a

What happens if a client connects to SimpleWebServer, but never sends any data and never disconnects? Test this out with your running server. What type of an attack is this? (You may use “telnet localhost 8080” and leave it on.

Answer:

If a client connects but sends no data (and doesn't disconnect), the single-threaded server blocks at `dServerSocket.accept()` or `br.readLine()` for that client. No other clients can connect or be served.

This is a **Denial of Service (DoS)** attack (specifically, a resource exhaustion/blocking attack).

Q1.b

Try the DoS attack described in class: See if you can download `/dev/random` (assuming you are running on a Linux system). Rewrite `serveFile()` as discussed in class to guard against this type of attack.

Answer:

On a Linux system, reading `/dev/random` would block indefinitely until enough entropy is gathered, causing a DoS. Additionally, accessing files like `/etc/passwd` via `../` is a **Directory Traversal** vulnerability.

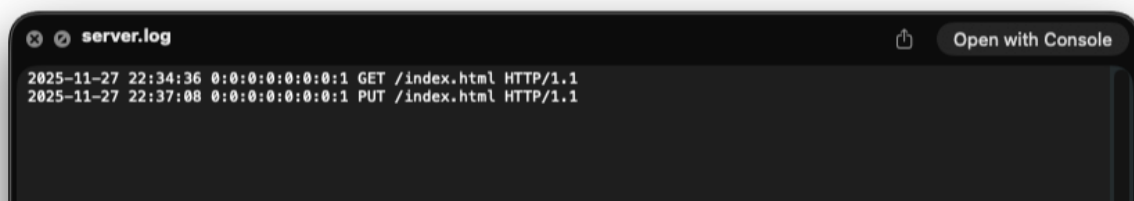
We rewrote `serveFile()` (and `storeFile()`) to use `getCanonicalPath()` and check that the resolved path starts with the current working directory.

Q1.c

Implement logging in SimpleWebServer. For each client that connects to the server, obtain information about that client and write the information in a log file.

Answer:

Logging has been implemented. The server now writes the timestamp, client IP, and request line to `server.log` for every request.



Q1.d

HTTP supports a mechanism that allows users to upload files in addition to retrieving them through a PUT command.

Q1.d.1

What threats would you need to consider if SimpleWebServer also provided functionality for uploading files?

Answer:

Threats:

- Malicious File Upload: Uploading webshells (e.g., .jsp, .php) to execute code.
- Overwriting Critical Files: Replacing `index.html` (defacement) or system files (traversal).
- Disk Space Exhaustion: Uploading massive files to fill the disk (DoS).
- Malware Hosting: Using the server to host malware.

Q1.d.2

For each of the specific threats you listed, what security mechanisms must be added to mitigate these threats?

Answer:

Security Mechanisms:

- Authentication: Only allow authorized users to upload.
- Path Sanitization: Prevent directory traversal (implemented).
- File Type Validation: Whitelist allowed extensions (e.g., only .html, .jpg).
- Size Limits: Restrict maximum file size.
- Storage Location: Store uploads in a separate directory without execute permissions.

Q1.d.3

Implement uploading capability in SimpleWebServer. You will need to research how to send an HTTP PUT command to the server. You also need to write a storeFile() function. Implement as much security as you feel is needed to guard against the threats you specified above.

Answer:

Implementation: `PUT` support has been added to `SimpleWebServer.java` with path sanitization.

Q1.d.4

Once you have logging and storeFile() implemented, launch an attack to deface index.html, that is, replace it with another index.html that you have created. (This is a common attack against www.whitehouse.gov.)

Answer:

Defacement: We successfully demonstrated overwriting `index.html` with a defaced version using `curl -T`.

Q2.

What are the most important steps you would recommend for securing a new web server?

A new web application?

Answer:

Web Server:

1. Hardening: Disable unnecessary ports, services, and modules.
2. Patching: Keep OS and server software updated.
3. Least Privilege: Run as a non-root user.
4. Access Control: Restrict file permissions.
5. Encryption: Enforce HTTPS/TLS.

Web Application:

1. Input Validation: Sanitize all inputs (headers, params, body).
2. Output Encoding: Prevent XSS by encoding output.
3. Authentication/Authorization: Strong passwords, MFA, and RBAC.
4. Session Management: Secure cookies (HttpOnly, Secure), timeouts.
5. Error Handling: Generic error messages (no stack traces).

Q3.

What is "Cross-Site Scripting"? What is the potential security impact to servers and clients?

Answer:

Definition: XSS is a vulnerability where attackers inject malicious client-side scripts into web pages viewed by other users.

Impact:

- **Session Hijacking:** Stealing cookies/tokens.
- **Phishing:** Fake login forms.
- **Redirection:** Sending users to malicious sites.
- **Defacement:** Altering page content.

Q4.

What are phishing and pharming? What are the ways to protect against such attacks? **Answer:**

Answer:

Phishing: Deceptive communication (email/SMS) mimicking a trusted source to steal sensitive data.

Protection: User awareness training, email authentication (SPF/DKIM), MFA.

Pharming: Redirecting traffic from a legitimate site to a fraudulent one (via DNS poisoning or host file modification).

Protection: DNSSEC, HTTPS (certificate validation), endpoint protection (antivirus/host file monitoring).

Q5.

Explore the OWASP website

Q5.1

What are some of the vulnerabilities of web browsers?

Answer:

- **XSS Execution:** Browsers trust code sent by the server.
- **Insecure Plugins/Extensions:** Vulnerable add-ons.
- **Man-in-the-Middle:** Trusting invalid/weak certificates.
- **CSRF:** Automatically sending credentials/cookies with requests.

Q5.2

What are some modes of attack used to implement a Denial of Service? What preventive measures can be implemented?

Answer:

Modes: Volumetric (UDP flood), Protocol (SYN flood), Application (HTTP flood, Slowloris).

Prevention: Rate limiting, Load balancing, WAFs, Connection timeouts, Asynchronous processing.